

8. Does Java support multiple inheritance?

No, it will not support for class

Yes, it will support for interface.

(03)

I want Yes or No : **NO**

9.6.25

Java version 1.8 Enhancement

(09)

V1.8

In Java version 1.8 two major changes were introduced in interface syntax

1. ~~Interface~~ static concrete methods.

2. default concrete methods.

eg) ①

interface Demo

{

static void test() // static concrete methods

}

System.out.println("Executing test()...");

}

public class Examples implements Demo

{

public static void main(String[] args)

{

Demo.test();

}

}

eg 2) interface Sample.

```
{  
    default void test() // default concrete method;  
    {  
        System.out.println("executing test()...");  
    }  
}
```

```
class Delta implements interface  
{  
  
}
```

```
public class Example2
```

```
{  
    public static void main(String[] args)  
    {  
        Sample ref = new Sample Delta();  
        ref.test();  
    }  
}
```

- These enhancement were added to interface syntax to provide functional programming capability to java programming language.

Functional Interface.

- Any interface in java that contains only one abstract methods is called functional interface.
- When declaring a functional interface it is recommended to use annotation (@) → @FunctionalInterface
- Functional interface was introduced in Java version 1.8 to support lambda expression
- Before version 1.8 an interface with only one abstract method was called as SAM (Single Abstract Method) type interface.

eg 1) @FunctionalInterface

interface A

{

void test();

}

eg 2)

interface B

extends A

{

void play();

}

eg 3)

interface Demo

{

void test();

}

class Sample implements Demo

{

@Override

public void test()

{

system.out.println("traditional implementation using overriding");

}

}

public class Mainclass

{

public static void main(String[] arg)

{

Demo d1 = new Sample();

d1.test();

System.out.println("-----");

Demo d2 = () -> { system.out.println("new way implementation using lambda expression"); }

};

d2.test();

}

}

Marker interface.

- An interface with an empty body is called as a marker interface.
- Marker interface does not directly provide any functionality to our program. its functionality is provided internally.
- Marker interface works like a flag and indicates support for a particular operation.
- Some of the examples of marker interface are

- * `java.lang.Cloneable`
- * `java.io.Serializable`
- * `java.util.RandomAccess`

eg 1. interface Playable

{

}

class cricket implements Playable

{

void play()

{

}

class Lagori

{

void play()

{

}

}

class olympics

{

~~public static void main(String[] args)~~

static void PlayGames (Playable p)

{

}

Access Modifiers

They are special modifiers used in Java programming to restrict visibility of members.

Java provides the following access modifiers.

1. private
2. default or package
3. protected
4. public

eg ~~package~~ com.jspiders.accessmodifier;

public class Fire

```
{
    private static int a = 10;
    static int b = 20;
    protected static int c = 30;
    public static int d = 40;

    p.s.v.m (String[] args)
    {
        s.o.pln (Fire.a);
        s.o.pln (Fire.b);
        s.o.pln (Fire.c);
        s.o.pln (Fire.d);
    }
}
```

~~package~~ import com.jspiders.accessmodifier;

public class Water

```
{
    p.s.v.m (String[] args)
    {
        s.o.pln (Fire.a); // error
        s.o.pln (Fire.b);
        s.o.pln (Fire.c);
        s.o.pln (Fire.d);
    }
}
```

package com.jspiders.def;
import com.jspiders.accessmodifier;
Fire
Oil extends Fire

public class Oil

```
{
    p.s.v.m (String[] args)
    {
        s.o.pln (Fire.a); // error
        s.o.pln (Fire.b); // error
        s.o.pln (Fire.c); ✓
        s.o.pln (Fire.d);
    }
}
```

private

Members with private access modifier can only be accessed

in the same class.

Default / package

Members with default or package level access can only be accessed in the same package.

A member is said to have default or package level access when the programmer fails to provide an access modifier.

Protected

- Members with protected access can be directly accessed in the same package.

- Access outside the package is controlled through inheritance.

Public

- Members with public access can be directly accessed from anywhere in the project.

ACCESS MODIFIER	RANKING	
	SECURITY	VISIBILITY
Private	1	4
Default/Package	2	3
Protected	3	2
Public	4	1

Access Modifiers for class/Interface/Enum.

- Whenever we declare a outer class or interface or enum it is only allowed to have public or default level access modifier.

overriding rules for access modifier.

- When overriding a method in ^{the} subclass we can retain the same access modifier as a super class or we can change to ^{an} ~~the~~ access modifier with higher visibility.

SUPER CLASS :

public

protected

<default/package>

SUB CLASS :

public

protected

<default/package>

public

protected

public

NOTE